C++ 中内存分配：栈与堆的清晰解析

在 C++ 里，**不是“某种数据类型天生就只能在栈/堆”**，而是**你怎么定义、怎么分配**，决定了它放在栈还是堆。

我直接按**实际写法**给你分清楚，一看就会。

一、默认放在 **栈内存** 的数据

只要你是**正常定义局部变量**，不管是什么类型，一律在栈上。

1. 基本数据类型（局部）

`int`、`char`、`float`、`double`、`bool`、`long` 等

Code block

```
void func() {
    int a = 10;     // 栈
    double b = 1.1; // 栈
    char c = 'x';   // 栈
}
```

2. 局部对象（类/结构体）

Code block

```
class Student {};

void func() {
    Student stu;  // 对象本身在栈上
}
```

3. 局部数组

Code block

```
void func() {
    int arr[100]; // 整个数组在栈上
}
```

4. 函数参数

Code block

```
void func(int x, string s) {
    // x 和 s 都在栈上
}
```

5. 指针变量**本身**

注意：**指针本身在栈，但它指向的内容不一定在栈！**

Code block

```
void func() {
    int* p;   // 指针变量 p 在栈上
}
```

二、一定放在 **堆内存** 的数据

只要你用了 `new` / `malloc` 动态分配，**无论什么类型，都在堆**。

1. new 出来的基本类型

Code block

```
int* p = new int(10); // 10 这块数据在堆
```

2. new 出来的对象

Code block

```
Student* pStu = new Student(); // 对象在堆
```

3. new 出来的动态数组

Code block

```
int* arr = new int[100]; // 数组在堆
```

三、特殊常见情况（非常重要）

1. vector、string 这类容器

- **容器对象本身：在栈**
- **内部真正存数据的缓冲区：在堆**

Code block

```
vector<int> v{1,2,3};
// v 本身在栈
// v 里面的 1,2,3 在堆
```

2. 全局变量 / static 变量

既不在栈，也不在堆，在**全局/静态存储区**。

程序一运行就存在，程序结束才释放。

四、一张表秒懂

写法	位置	例子
局部普通变量	栈	int a;
局部对象/数组	栈	A a; int arr[5];
函数参数	栈	void f(int x)
指针变量本身	栈	int* p;
new/malloc 分配	堆	new int; new A();
vector/string 内部数据	堆	vector<int> v
全局/static 变量	静态区	global int a; static int b

五、最容易错的一句话

指针本身在栈，指针指向的内容可能在堆。

Code block

```
int* p = new int(5);
// p 在栈
// *p（那个 5）在堆
```

极简总结

- **栈**：局部变量、函数参数、普通数组、对象、指针本身
- **堆**：`new` / `malloc` 出来的一切、容器内部数据
- **既不是栈也不是堆**：全局变量、静态变量

（注：文档部分内容可能由 AI 生成）